In [1]:
```
import numpy as np
import sklearn as sk
import matplotlib
import matplotlib.pyplot as plt
import scipy.stats
from sklearn import metrics
from sklearn.linear_model import LogisticRegression
import pandas as pd
```

In [2]:
```
#Set the default figure size for the notebook, so they are not all tiny
matplotlib.rcParams['figure.figsize'] = (12,8)
```

In [3]:
```
spam = pd.read_csv('ElemStatLearn_SpamData.csv')
spam.shape
```

Out[3]:
```
(4601, 58)
```

In [4]:
```
print(spam.shape)
spam.head()
```

```
(4601, 58)
```

Out[4]:

	word_freq_make	word_freq_address	word_freq_all	word_freq_3d	word_freq_our	word_freq_ove
0	0.00	0.64	0.64	0.0	0.32	0.0
1	0.21	0.28	0.50	0.0	0.14	0.2
2	0.06	0.00	0.71	0.0	1.23	0.1
3	0.00	0.00	0.00	0.0	0.63	0.0
4	0.00	0.00	0.00	0.0	0.63	0.0

5 rows × 58 columns

In [5]:
```
X = spam.iloc[:,0:57]
y = np.where(spam['spam']=='spam',1,0)
```

In [6]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, ran
```

In [7]:
```
#The sklearn version of logistic regression automatically has a ridge penalty
#Large C means small penalty here (opposite of lambda)
lmfit = LogisticRegression(C=1000000).fit(X_train, y_train)

yhat_lm = lmfit.predict(X_test)
np.mean(y_test!=yhat_lm)
```

```
/home/maxg/.pyenv/versions/3.9.0/lib/python3.9/site-packages/sklearn/linear_m
odel/_logistic.py:762: ConvergenceWarning: lbfgs failed to converge (status=
1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
```

```
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres
sion
  n_iter_i = _check_optimize_result(
```

Out[7]: 0.06978275181040158

In [8]:
```
from sklearn.linear_model import LogisticRegressionCV
lassofit = LogisticRegressionCV(penalty='l1', solver='liblinear', Cs=50).fit(
yhat_lasso = lassofit.predict(X_test)
np.mean(y_test!=yhat_lasso)
```

Out[8]: 0.07570770243581304

How about trees?

In [9]:
```
from sklearn.tree import DecisionTreeClassifier
treefit = DecisionTreeClassifier(random_state=0).fit(X_train,y_train)
yhat_tree = treefit.predict(X_test)
np.mean(yhat_tree != y_test)
```

Out[9]: 0.09084924292297564

In [10]:
```
from sklearn.tree import DecisionTreeClassifier
treefit = DecisionTreeClassifier(min_samples_leaf=200).fit(X_train,y_train)
yhat_tree = treefit.predict(X_test)
np.mean(yhat_tree != y_test)
```

Out[10]: 0.14351547070441079

In [11]:
```
from sklearn.tree import DecisionTreeClassifier
treefit = DecisionTreeClassifier(min_samples_leaf=20).fit(X_train,y_train)
yhat_tree = treefit.predict(X_test)
np.mean(yhat_tree != y_test)
```

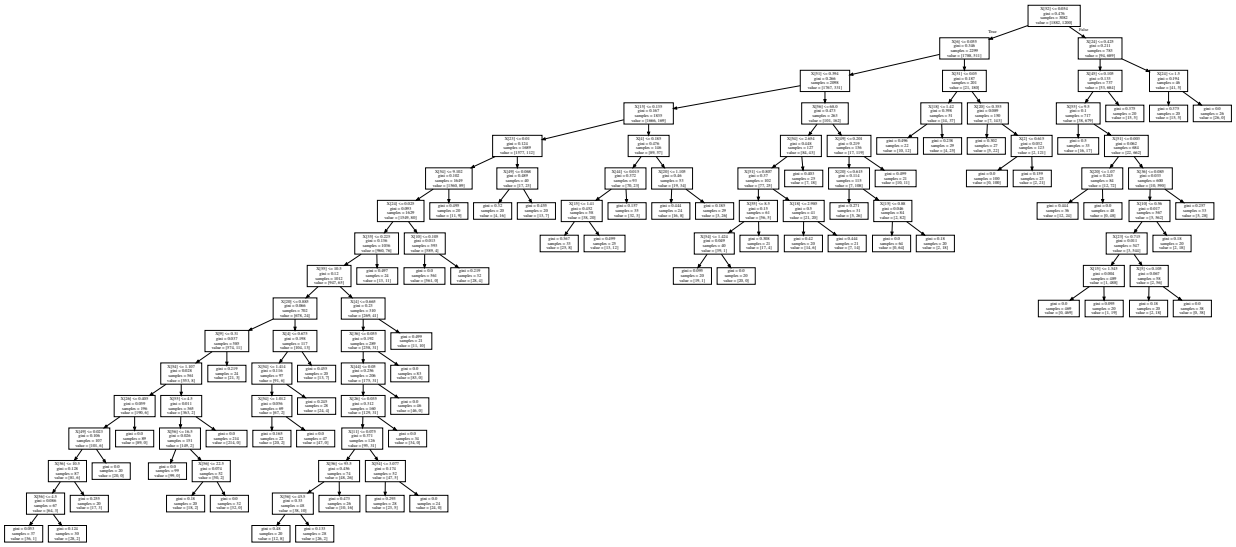
Out[11]: 0.10204081632653061

In [12]:
```
import graphviz
from sklearn.tree import export_graphviz
```

In [ ]:
```
```

In [13]:
```
dot_data = export_graphviz(treefit, out_file=None)
graph = graphviz.Source(dot_data)
graph
```

Out[13]:

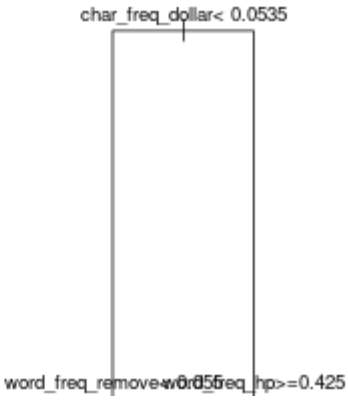


What happens if we do pruning in R?

In [14]:
```
import rpy2
%load_ext rpy2.ipython
```

In [15]:
```
%%R -i X_train -i X_test -i y_train -i y_test

#You can also use the library "tree", but it is not as nice
library(rpart)
dat_train = cbind(as.factor(y_train),X_train)
dat_test = cbind(as.factor(y_test),X_test)
names(dat_train)[1] = 'y'
names(dat_test)[1] = 'y'

#Note: I'm overriding the control parameters so you can see a bigger tree.
#In reality, the defaults are a smarter bet.
tree = rpart(y~., data=dat_train, control=rpart.control(cp=0.0001, minbucket=

yhat1 = predict(tree, dat_test, type='class')
print(mean(yhat1 != y_test))

plot(tree)
text(tree, pretty=0, cex=0.8)
```

```
[1] 0.08689928
```



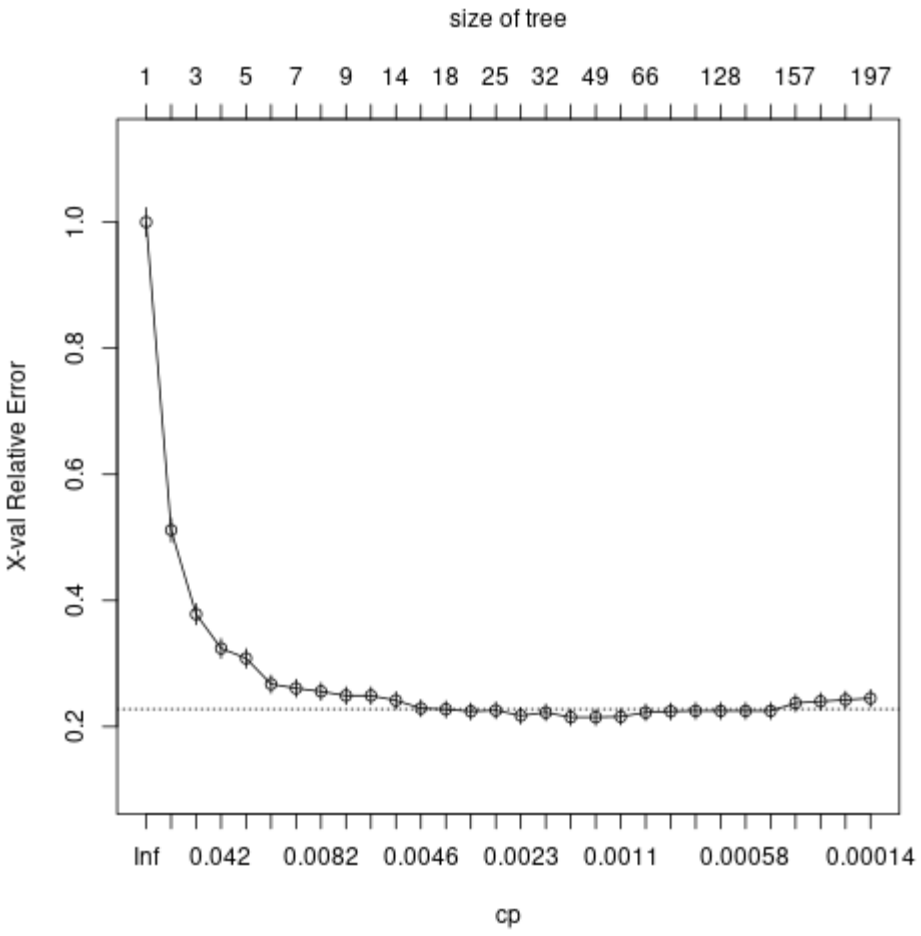
In [16]:
```
%%R

printcp(tree)
```

```
Classification tree:
rpart(formula = y ~ ., data = dat_train, control = rpart.control(cp = 1e-04,
    minbucket = 1))

Variables actually used in tree construction:
 [1] capital_run_length_average capital_run_length_longest
 [3] capital_run_length_total   char_freq_bracket
 [5] char_freq_dollar           char_freq_exc
 [7] char_freq_paren            word_freq_000
 [9] word_freq_1999             word_freq_650
[11] word_freq_address          word_freq_addresses
[13] word_freq_all              word_freq_business
[15] word_freq_conference       word_freq_credit
[17] word_freq_data             word_freq_edu
[19] word_freq_email            word_freq_font
[21] word_freq_free             word_freq_george
[23] word_freq_hp               word_freq_hpl
[25] word_freq_internet         word_freq_labs
[27] word_freq_mail             word_freq_make
[29] word_freq_meeting          word_freq_money
[31] word_freq_order            word_freq_original
[33] word_freq_our              word_freq_over
[35] word_freq_people           word_freq_pm
[37] word_freq_project          word_freq_re
[39] word_freq_receive          word_freq_remove
[41] word_freq_report           word_freq_technology
[43] word_freq_telnet           word_freq_will
[45] word_freq_you              word_freq_your

Root node error: 1200/3082 = 0.38936

n= 3082

           CP nsplit rel error  xerror     xstd
1  0.49583333      0  1.000000 1.00000 0.022558
2  0.13250000      1  0.504167 0.51167 0.018478
3  0.05083333      2  0.371667 0.37833 0.016396
4  0.03416667      3  0.320833 0.32417 0.015364
5  0.03000000      4  0.286667 0.30833 0.015037
```

```
6  0.01166667      5  0.256667 0.26750 0.014131
7  0.01000000      6  0.245000 0.26083 0.013974
8  0.00666667      7  0.235000 0.25583 0.013855
9  0.00625000      8  0.228333 0.24917 0.013693
10 0.00583333     11  0.207500 0.24917 0.013693
11 0.00500000     13  0.195833 0.24167 0.013507
12 0.00416667     14  0.190833 0.22917 0.013188
13 0.00375000     17  0.178333 0.22750 0.013145
14 0.00333333     19  0.170833 0.22417 0.013058
15 0.00250000     24  0.154167 0.22583 0.013101
16 0.00208333     28  0.144167 0.21750 0.012880
17 0.00200000     31  0.137500 0.22250 0.013014
18 0.00166667     36  0.127500 0.21500 0.012813
19 0.00125000     48  0.105833 0.21500 0.012813
20 0.00092593     56  0.095833 0.21583 0.012835
21 0.00083333     65  0.087500 0.22250 0.013014
22 0.00069444    112  0.048333 0.22417 0.013058
23 0.00062500    118  0.044167 0.22500 0.013080
24 0.00059524    127  0.038333 0.22500 0.013080
25 0.00055556    135  0.033333 0.22500 0.013080
26 0.00041667    138  0.031667 0.22500 0.013080
27 0.00035714    156  0.024167 0.23750 0.013402
28 0.00027778    163  0.021667 0.24000 0.013465
29 0.00020833    192  0.011667 0.24250 0.013528
30 0.00010000    196  0.010833 0.24500 0.013590
```

In [17]:

```
%%R

plotcp(tree)
```

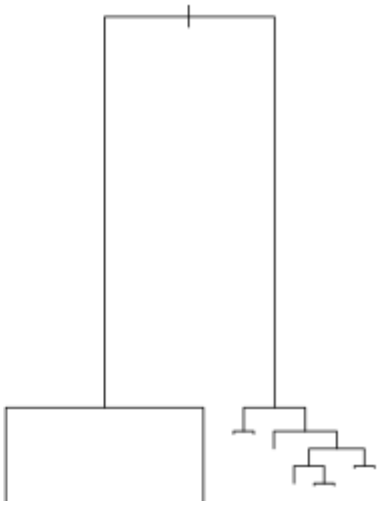


In [18]:
```
%%R

mincp = tree$cptable[which.min(tree$cptable[,"xerror"]),"CP"]

mincp
```

```
[1] 0.001666667
```

In [19]:
```
%%R

tree_pruned = prune(tree, cp = mincp)

plot(tree_pruned)
#text(tree_pruned)
yhat1 = predict(tree_pruned, dat_test, type='class')
print(mean(yhat1 != y_test))
```

```
[1] 0.09019092
```



Finally, let's look at random forests

In [20]:
```
from sklearn.ensemble import RandomForestClassifier
```

In [21]:
```
fitrf = RandomForestClassifier(n_estimators=500, oob_score=True, random_state
```

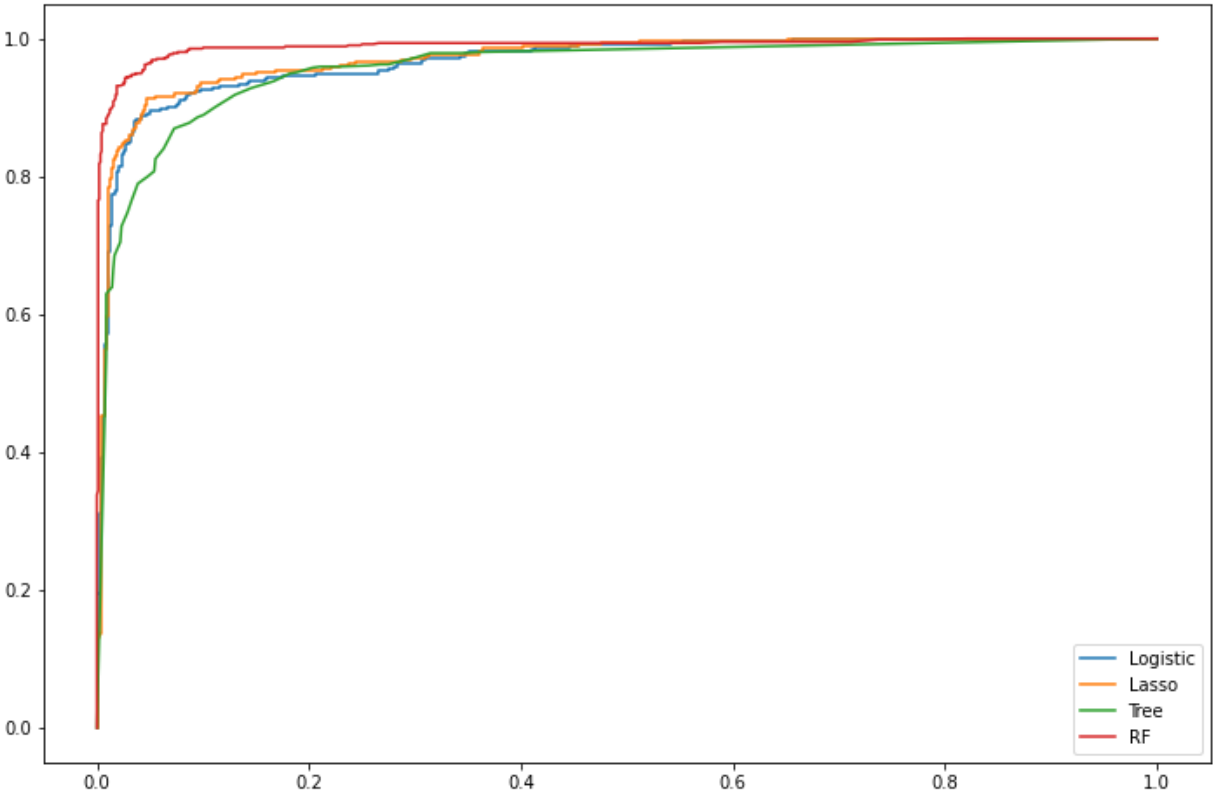
In [22]:
```
yhat_rf = fitrf.predict(X_test)
np.mean(yhat_rf != y_test)
```

Out[22]:
```
0.040157998683344305
```

In [23]:
```
1-fitrf.oob_score_
```

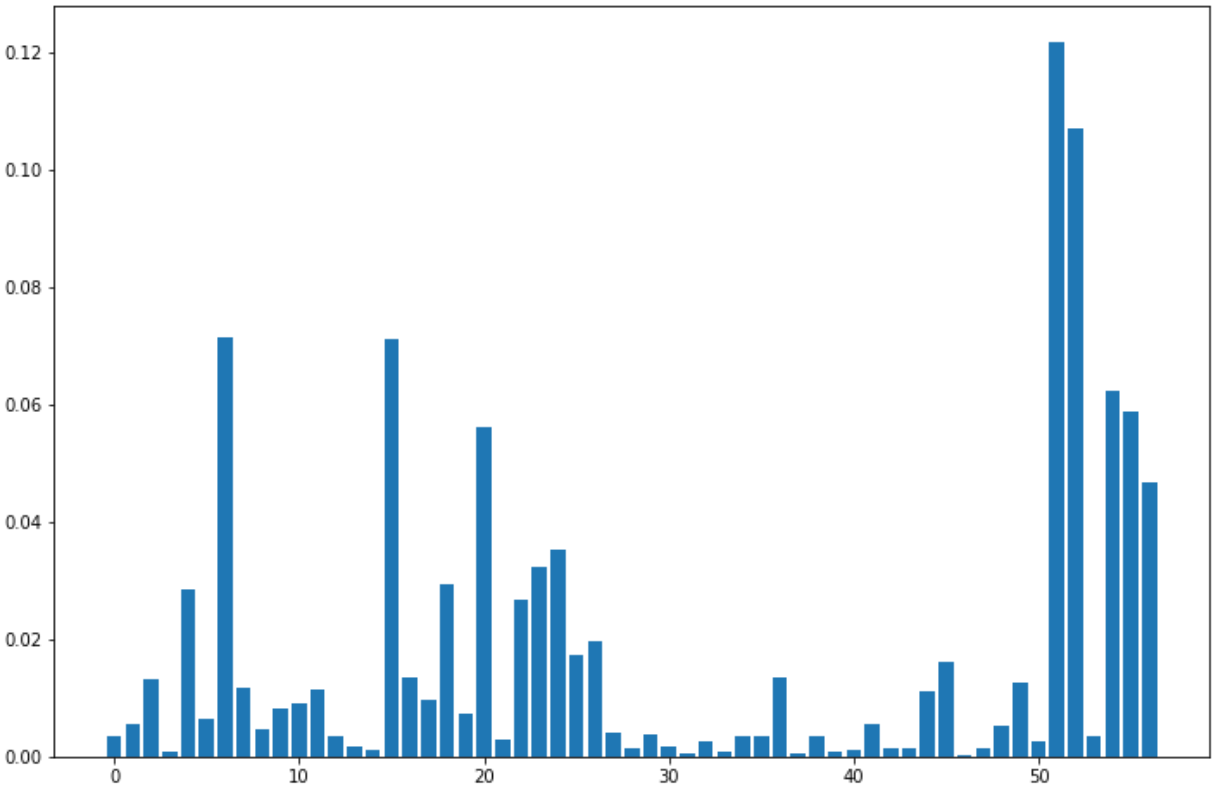
Out[23]:
```
0.053212199870214194
```

In [24]:
```
from sklearn.metrics import roc_curve

roc_logit = roc_curve(y_test,  lmfit.predict_proba(X_test)[:,1])
roc_lasso = roc_curve(y_test, lassofit.predict_proba(X_test)[:,1])
roc_tree = roc_curve(y_test, treefit.predict_proba(X_test)[:,1])
roc_rf = roc_curve(y_test, fitrf.predict_proba(X_test)[:,1])

line1, = plt.plot(roc_logit[0],roc_logit[1])
line2, = plt.plot(roc_lasso[0],roc_lasso[1])
line3, = plt.plot(roc_tree[0],roc_tree[1])
line4, = plt.plot(roc_rf[0],roc_rf[1])

plt.legend((line1,line2,line3,line4), ('Logistic','Lasso','Tree','RF'))
plt.show()
```



How does it work?

In [25]:
```
fitrf.feature_importances_
plt.bar(range(X.shape[1]), fitrf.feature_importances_)
plt.show()
```



In [26]:
```
a = list(zip(X_train.columns, fitrf.feature_importances_))
a.sort(key=lambda x: x[1], reverse=True)
a
```

Out[26]:
```
[('char_freq_exc', 0.12179566660436655),
 ('char_freq_dollar', 0.1070563165310063),
 ('word_freq_remove', 0.0714860093519845),
 ('word_freq_free', 0.07110568543290978),
 ('capital_run_length_average', 0.06239446008509335),
 ('capital_run_length_longest', 0.058634945635547726),
 ('word_freq_your', 0.05622910726334724),
 ('capital_run_length_total', 0.046608385450272544),
 ('word_freq_hp', 0.035223768566802864),
 ('word_freq_money', 0.032228501995478685),
 ('word_freq_you', 0.029278247831878886),
 ('word_freq_our', 0.028522402809727163),
 ('word_freq_000', 0.02675308052504121),
 ('word_freq_george', 0.019741632925250895),
 ('word_freq_hpl', 0.017169316004847116),
 ('word_freq_edu', 0.016096834876735563),
 ('word_freq_1999', 0.013551987361549299),
 ('word_freq_business', 0.01349423438241483),
 ('word_freq_all', 0.013208110352906975),
 ('char_freq_paren', 0.012470055462562966),
 ('word_freq_internet', 0.011691793091055057),
 ('word_freq_will', 0.011369644757454547),
 ('word_freq_re', 0.010968852170980918),
 ('word_freq_email', 0.009726662262082799),
 ('word_freq_receive', 0.009045790680224354),
 ('word_freq_mail', 0.00805785409940942),
 ('word_freq_credit', 0.007255535596738994),
 ('word_freq_over', 0.006409772038351993),
 ('word_freq_address', 0.005620849807606672),
 ('word_freq_meeting', 0.005620773228862744),
 ('char_freq_semicol', 0.005159512993676977),
 ('word_freq_order', 0.004718552596218803 5),
 ('word_freq_650', 0.003912727156780143),
 ('word_freq_labs', 0.0038071629183715422),
 ('char_freq_pound', 0.0035914921460870005),
 ('word_freq_technology', 0.0035210739110632335),
 ('word_freq_85', 0.003466071955572478),
 ('word_freq_make', 0.0034264089561055395),
 ('word_freq_people', 0.0034087915241643517),
 ('word_freq_pm', 0.0033657932312073837),
 ('word_freq_font', 0.002888080257790889),
 ('char_freq_bracket', 0.002689774625121385),
 ('word_freq_data', 0.0024627569505202463),
 ('word_freq_report', 0.0017550759147596867),
 ('word_freq_telnet', 0.0016848413447021187),
 ('word_freq_lab', 0.001540398620816898),
 ('word_freq_original', 0.001496690298740242),
 ('word_freq_conference', 0.0013196518552613648),
 ('word_freq_project', 0.001270164586762057),
 ('word_freq_addresses', 0.0011293369678016642),
 ('word_freq_cs', 0.0010733523838042836),
 ('word_freq_direct', 0.0008463255223699379),
 ('word_freq_3d', 0.0007430916159058321),
 ('word_freq_415', 0.0006970848729959653),
```

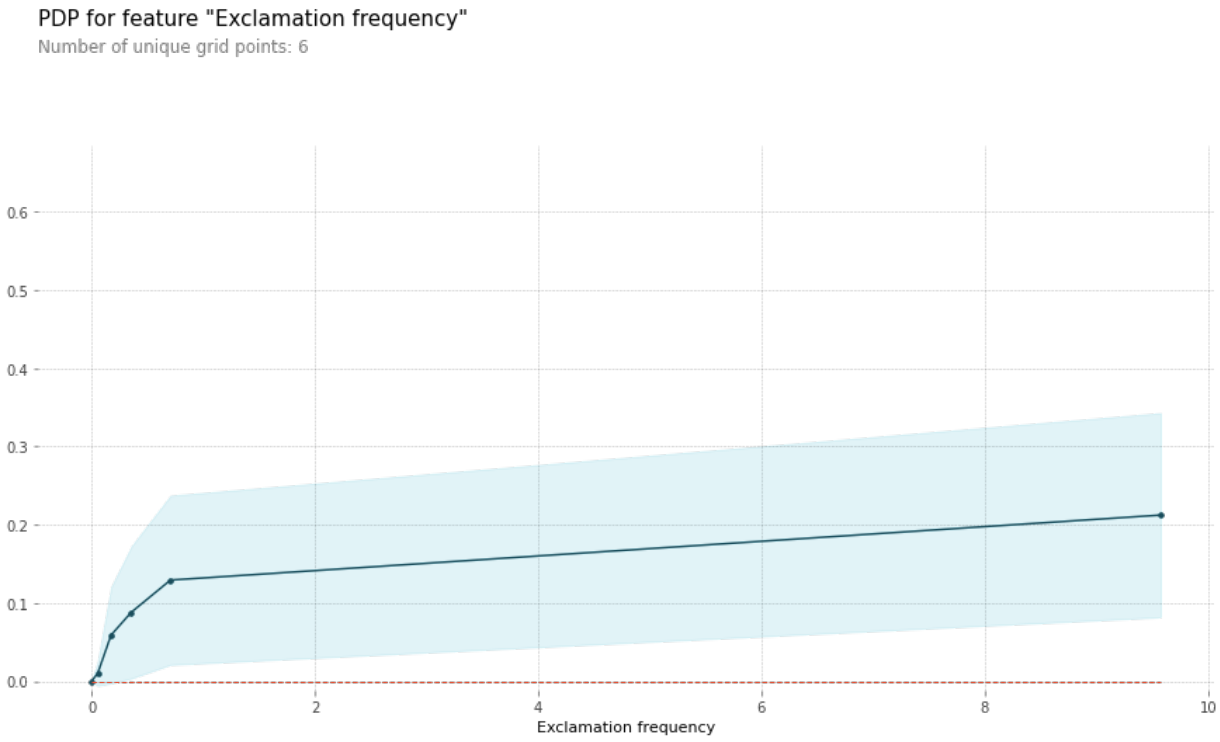
```
('word_freq_857', 0.0005023001588100264),
('word_freq_parts', 0.0004464618458292403),
```

In [27]:
```
from pdpbox import pdp
```

In [28]:
```
pdpobj = pdp.pdp_isolate(fitrf, X_test, X_test.columns, 'char_freq_exc')
```

In [29]:
```
pdp.pdp_plot(pdpobj, 'Exclamation frequency')
plt.show()
```

```
findfont: Font family ['Arial'] not found. Falling back to DejaVu Sans.
findfont: Font family ['Arial'] not found. Falling back to DejaVu Sans.
findfont: Font family ['Arial'] not found. Falling back to DejaVu Sans.
findfont: Font family ['Arial'] not found. Falling back to DejaVu Sans.
```



Or in R:

In [30]:
```
%%R -i X_train -i X_test -i y_train -i y_test

library(randomForest)

fitrf = randomForest(X_train, as.factor(y_train))
fitrf
```

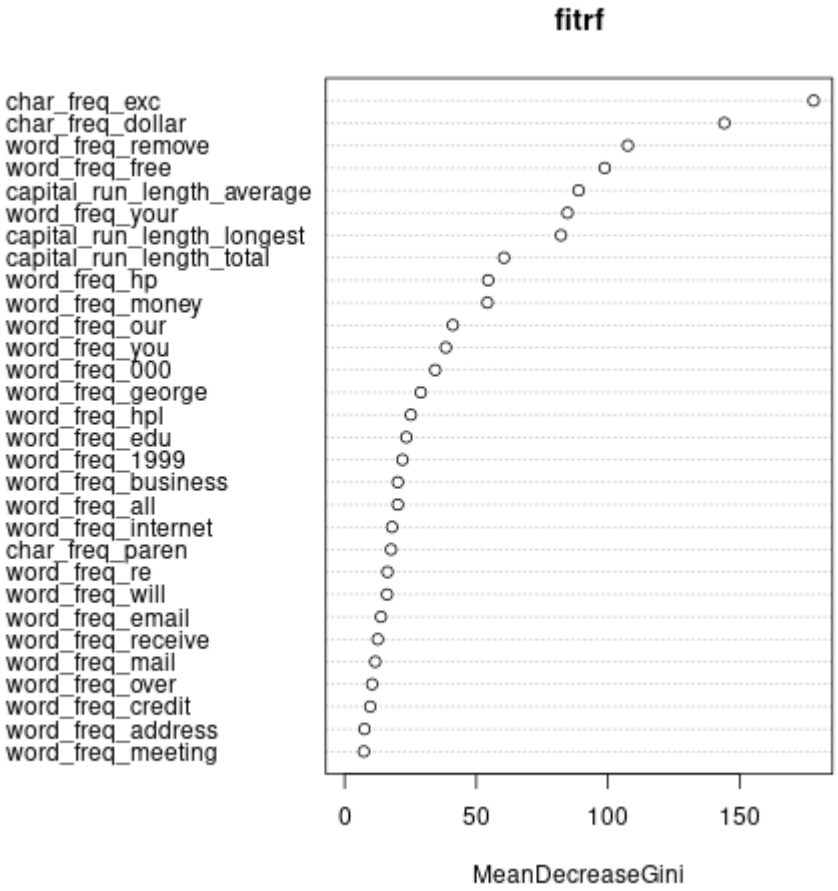
```
R[write to console]: randomForest 4.6-14

R[write to console]: Type rfNews() to see new features/changes/bug fixes.

Call:
 randomForest(x = X_train, y = as.factor(y_train))
               Type of random forest: classification
                     Number of trees: 500
```

```
No. of variables tried at each split: 7

        OOB estimate of  error rate: 5.22%
Confusion matrix:
     0    1 class.error
0 1826   56  0.02975558
1  105 1095  0.08750000
```

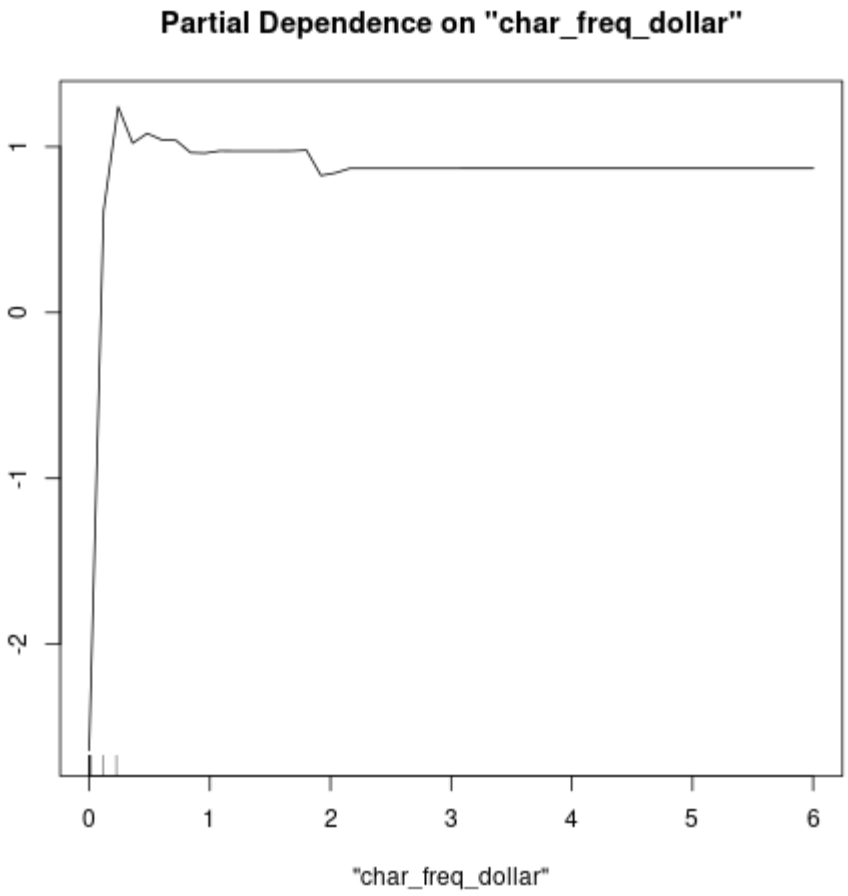
In [31]:
```
%%R
mean(predict(fitrf, X_test)!= y_test)
```

```
[1] 0.04344964
```

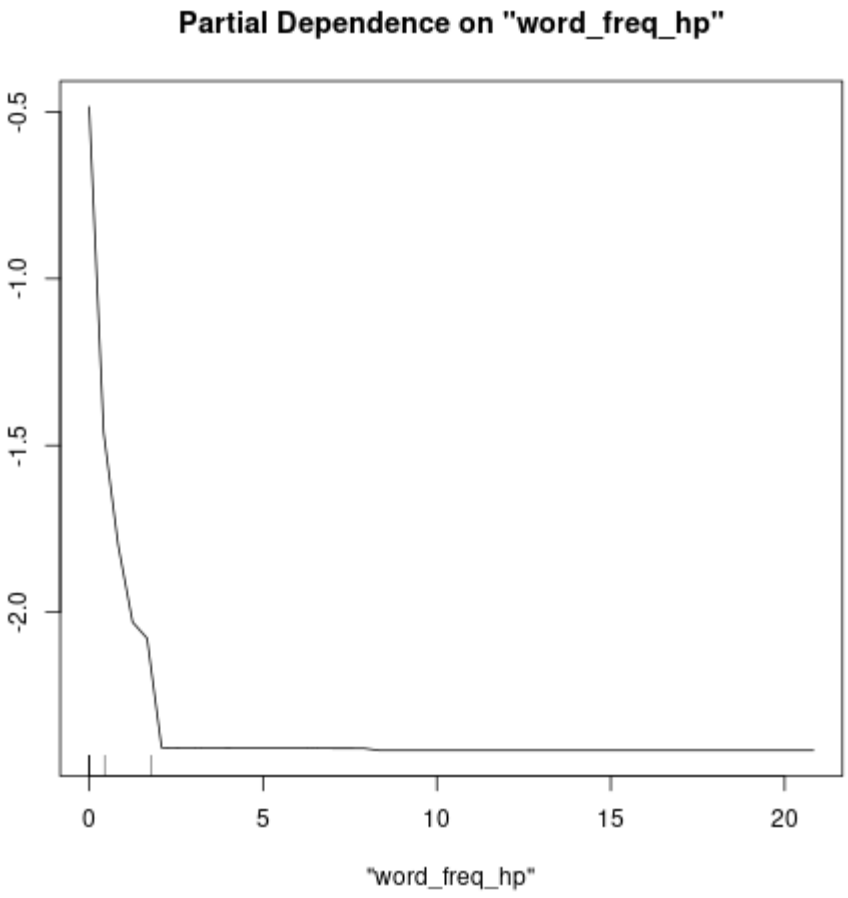
In [32]:
```
%%R
varImpPlot(fitrf)
```



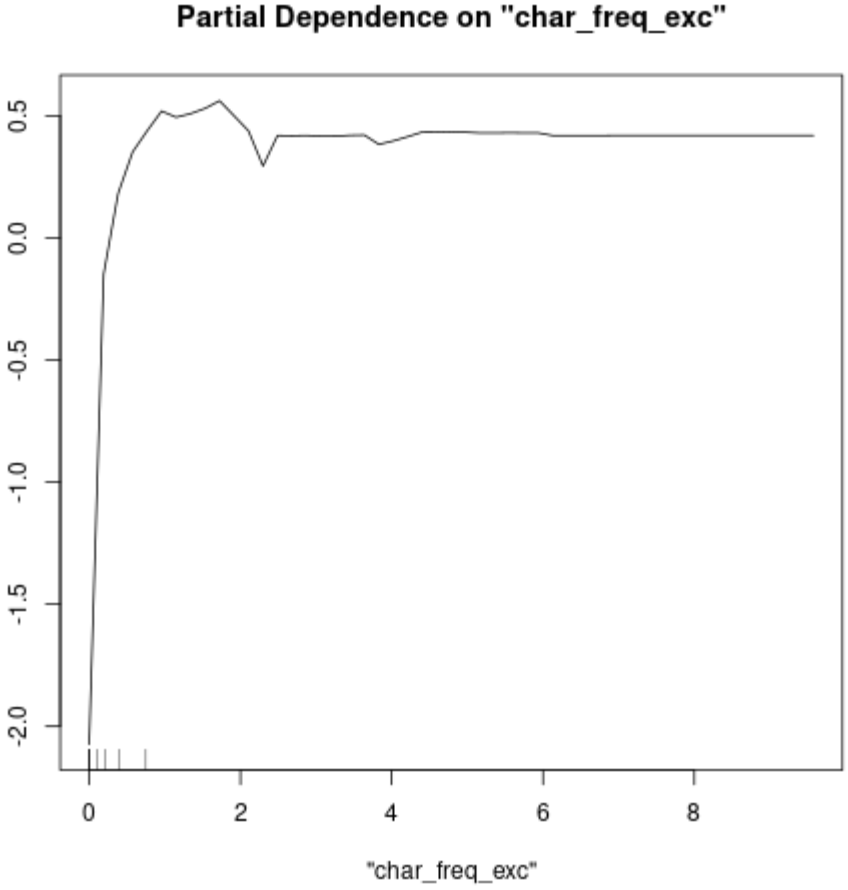
In [33]:
```
%%R
partialPlot(fitrf, X_train, x.var='char_freq_dollar', which.class=1)
```



In [34]:
```
%%R
partialPlot(fitrf, X_train, x.var='word_freq_hp', which.class=1)
```



In [35]:
```
%%R
partialPlot(fitrf, X_test, x.var='char_freq_exc', which.class=1)
```


Partial Dependence on "char_freq_exc"
0.5
0.0
-0.5
-1.0
-1.5
-2.0
0
2
4
6
8
"char_freq_exc"